Cache Hierarchy and Vectorization Analysis of Lindblad Master Equation Simulation for Near-Term Quantum Control

Rylan Malarchick[a,*]

[a] *Department of Engineering Physics, Embry-Riddle Aeronautical University, Daytona Beach, FL, 32114, USA*

Abstract

Simulation of open quantum systems via the Lindblad master equation is a computational bottleneck in near-term quantum control workflows, including optimal pulse engineering (GRAPE), trajectory-based robustness analysis, and feedback controller design. For the system sizes relevant to near-term quantum control ($d = 3$ for a single transmon with leakage, $d = 9$ for two-qubit, and $d = 27$ for three-qubit), the dominant cost per timestep is a ($d^2 \times d^2$) complex matrix-vector multiplication: a 9×9, 81×81, or 729×729 dense matvec, respectively. The working set sizes (1.5 KB, 105 KB, and 8.1 MB) straddle the L1, L2, and L3 cache boundaries of modern CPUs, making this an ideal system for cache-hierarchy performance analysis. We characterize the arithmetic intensity ($\approx 1/2$ FLOP/byte in the large-d limit), construct a Roofline model for the propagation kernel, and systematically vary compiler flags and data layout to isolate the contributions of auto-vectorization, fused multiply-add, and structure-of-arrays (SoA) memory layout. We show that SoA layout combined with `-O3 -march=native -ffast-math` yields 2–4× speedup over scalar array-of-structures baselines, and that `-ffast-math` is essential for enabling GCC auto-vectorization of complex arithmetic. These results motivate a set of concrete recommendations for authors of scientific simulation libraries that operate on small, fixed-size dense complex matrices.

Keywords: Lindblad master equation, cache hierarchy, Roofline model, SIMD vectorization, quantum simulation, structure-of-arrays

1. Introduction

Dense matrix-vector multiplication on small, fixed-size complex matrices is a ubiquitous computational kernel in physics simulation. Examples include density matrix propagation in quantum optics, transfer matrix methods in condensed matter, and Jones calculus in polarization optics. When the matrix dimension n is small enough that the working set fits in CPU cache (typically $n \lesssim 1000$), the performance characteristics differ qualitatively from the large-matrix regime studied in classical BLAS benchmarking: the kernel is no longer DRAM-bandwidth-limited but instead sensitive to cache-level bandwidth, data layout, and compiler vectorization decisions. To our knowledge, no prior work has performed a Roofline-style microarchitectural analysis of dense complex matvec at these small, fixed sizes. This paper fills that gap using the Lindblad master equation as a concrete and practically important test case.

The Lindblad master equation [1, 2] governs the time evolution of an open quantum system:

$$\frac{d\rho}{dt} = -i[H, \rho] + \sum_k \Big(L_k \rho L_k^\dagger - \tfrac{1}{2}\{L_k^\dagger L_k, \rho\} \Big), \qquad (1)$$

where ρ is the $d \times d$ density matrix, H is the system Hamiltonian, and $\{L_k\}$ are the collapse operators encoding decoherence. For a single transmon qubit modeled as a three-level system ($d = 3$) [3], Eq. (1) must be integrated thousands of times per optimization iteration in GRAPE-based pulse engineering [4], and millions of times across full robustness atlases [5].

Vectorizing Eq. (1) by writing $\vec{\rho} \equiv \mathrm{vec}(\rho) \in \mathbb{C}^{d^2}$ yields the linear system

$$\frac{d\vec{\rho}}{dt} = \mathcal{L}\vec{\rho}, \qquad \mathcal{L} \in \mathbb{C}^{d^2 \times d^2}, \qquad (2)$$

whose formal solution is $\vec{\rho}(t) = e^{\mathcal{L}t}\vec{\rho}(0)$. For time-independent systems, the propagator $P = e^{\mathcal{L}\Delta t}$ can be precomputed once and applied at every timestep as a

*Corresponding author
Email address: `malarchr@my.erau.edu` (Rylan Malarchick)

dense matvec:

$$\vec{\rho}(t + \Delta t) = P\,\vec{\rho}(t), \qquad P \in \mathbb{C}^{d^2 \times d^2}. \tag{3}$$

The dominant per-step cost is Eq. (3): a $(d^2) \times (d^2)$ complex matrix-vector product. At $d = 3$ this is 9×9; at $d = 9$, 81×81; at $d = 27$, 729×729. This paper asks: *where does this computation sit on the memory hierarchy, what does the compiler actually generate, and how much performance is left on the table by standard reference implementations?*

The dominant open-source tool for Lindblad simulation is QuTiP [6, 7], which wraps SciPy sparse solvers in a Python interface. Recent JAX-based tools such as dynamiqs [8] offer GPU acceleration and automatic differentiation for gradient-based control. However, for the small, dense systems central to near-term quantum control ($d \leq 27$), these frameworks carry significant interpreter and dispatch overhead relative to the arithmetic. This motivates a bare-metal C implementation whose performance can be analyzed at the microarchitectural level.

2. Background

2.1. The Lindblad propagation kernel

The vectorized Lindbladian $\mathcal{L}$ in Eq. (2) is constructed via Kronecker products. We use the row-major vectorization convention, where $\mathrm{vec}(\rho)$ stacks the rows of ρ into a column vector. Under this convention,

$$\mathrm{vec}(A\rho B) = (A \otimes B^T)\ \mathrm{vec}(\rho). \tag{4}$$

Applying this identity to each term of Eq. (1) yields the coherent part:

$$\mathrm{vec}(-i[H, \rho]) = -i(H \otimes I - I \otimes H^T)\ \mathrm{vec}(\rho), \tag{5}$$

and the dissipative part (for each collapse operator L_k):

$$\mathrm{vec}(L_k \rho L_k^\dagger) = (L_k \otimes L_k^*)\ \mathrm{vec}(\rho), \tag{6}$$

$$\mathrm{vec}(L_k^\dagger L_k\, \rho) = (L_k^\dagger L_k \otimes I)\ \mathrm{vec}(\rho), \tag{7}$$

$$\mathrm{vec}(\rho\, L_k^\dagger L_k) = (I \otimes (L_k^\dagger L_k)^T)\ \mathrm{vec}(\rho), \tag{8}$$

where $L_k^* \equiv \overline{L_k}$ denotes the element-wise complex conjugate and we used $(L_k^\dagger)^T = L_k^*$. Collecting terms:

$$\begin{aligned}\mathcal{L} = &-i(H \otimes I - I \otimes H^T) \\ &+ \sum_k \Big[L_k \otimes L_k^* - \tfrac{1}{2}(L_k^\dagger L_k \otimes I) \\ &\qquad - \tfrac{1}{2}(I \otimes (L_k^\dagger L_k)^T) \Big]. \end{aligned} \tag{9}$$

For Hermitian H, $H^T = H^*$. The full Lindbladian is a $d^2 \times d^2$ dense complex matrix, constructed once and cached as the propagator $P = e^{\mathcal{L}\Delta t}$.

Table 1: Working set sizes and cache-level placement for the propagation kernel on i9-13980HX (48 KB L1d, 2 MB L2, 36 MB L3).

d	d^2	Hot WS	Cache	AI
3	9	1.5 KB	L1	0.41
9	81	105 KB	L2	0.49
27	729	8.3 MB	L3	0.50

2.2. Cache hierarchy and the Roofline model

The Roofline model [9] characterizes the performance of a computational kernel in terms of its *arithmetic intensity* (FLOP/byte) relative to a machine's compute and bandwidth ceilings. A kernel is *compute-bound* if its arithmetic intensity exceeds the *ridge point* $I^* = \Pi/\beta$ (peak FLOP/s divided by peak bandwidth GB/s); otherwise it is *memory-bound*.

For the propagation matvec (Eq. (3)):

$$\text{FLOPs} = 8\,d^4 \quad \text{(complex multiply-add)}, \tag{10}$$

$$\text{bytes} = (d^4 + 2d^2) \times 16 \quad \text{(complex128)}, \tag{11}$$

$$\text{AI} = \frac{8\,d^4}{(d^4 + 2d^2) \times 16} \xrightarrow{d\to\infty} \frac{1}{2}\ \text{FLOP/byte}. \tag{12}$$

For a modern CPU with ridge point $I^* \approx 1$–2 FLOP/byte, the propagation kernel is *always memory-bound* at these system sizes.

Table 1 summarizes the working set sizes for the three target system dimensions. The hot working set for each propagation step consists of the propagator matrix P (d^4 complex elements), the input state vector (d^2 elements), and the output state vector (d^2 elements), each stored as 16-byte `complex128`.

All three operating points have AI well below the DRAM ridge point $I^* = 128/80 = 1.6$ FLOP/byte, confirming the memory-bound prediction.

2.3. Compiler auto-vectorization of complex matvec

Modern x86-64 CPUs implement 256-bit SIMD through AVX2, with each `ymm` register holding four 64-bit doubles. For complex arithmetic, two storage layouts are natural.

In *Array-of-Structures* (AoS) layout, each complex number is stored as a (real, imaginary) pair: the `double complex` type in C. A `ymm` register holds two complex numbers: $[\mathrm{re}_0, \mathrm{im}_0, \mathrm{re}_1, \mathrm{im}_1]$. Complex multiplication in AoS layout requires lane-crossing shuffles (`vpermilpd`) to separate real and imaginary parts before multiplying.

In *Structure-of-Arrays* (SoA) layout, real and imaginary parts are stored in separate contiguous arrays. A `ymm` register holds four consecutive real (or imaginary)

values: $[\mathrm{re}_0, \mathrm{re}_1, \mathrm{re}_2, \mathrm{re}_3]$. This enables the compiler to vectorize the inner product loop cleanly: four multiplications `P_re[j:j+4] * v_re[j:j+4]` map directly to a single `vmulpd` instruction with no shuffle overhead.

A critical compiler consideration is that GCC's default complex multiplication calls the library function `__muldc3`, which handles NaN and infinity according to Annex G of the C standard. This function call defeats auto-vectorization entirely. The flag `-ffast-math` (specifically, `-fcx-limited-range`) permits the compiler to inline direct multiply–add/subtract sequences, enabling both fused multiply-add (FMA) and SIMD vectorization [10].

With `-ffast-math` enabled for AoS layout, GCC emits `vfmsubadd231pd` [11]: this instruction multiplies two `ymm` registers and alternately subtracts and adds the products to a third register, precisely the pattern needed for complex multiply-accumulate. For SoA layout, the compiler emits standard `vmulpd` and `vfmadd231pd`/`vfmsub231pd` on four-wide vectors of pure real or pure imaginary values.

3. Methods

3.1. C library implementation

We implement the Lindblad propagator in ISO C11 with no external dependencies. The library provides:

- `lb_build_lindbladian`: constructs $\mathcal{L}$ via Kronecker products (Eq. (9));
- `lb_expm`: computes $P = e^{\mathcal{L}\Delta t}$ via Padé [13/13] with scaling and squaring [12];
- `lb_propagate_step`: the hot-path AoS matvec (Eq. (3));
- `lb_propagate_step_soa`: SoA-layout matvec;
- `lb_propagate_step_avx2`: hand-written AVX2 intrinsics (AoS layout);
- `lb_evolve`: full trajectory integration.

The source code is available at `https://github.com/rylanmalarchick/lindblad-bench` under the MIT license.

3.2. Compiler flag experiments

We build `lb_propagate_step` under four flag configurations using GCC 13.3 on x86-64 (Intel Core i9-13980HX, AVX2):

1. `-O2 -fno-tree-vectorize`: scalar baseline;
2. `-O3`: basic auto-vectorization;
3. `-O3 -march=native`: AVX2 auto-vectorization;
4. `-O3 -march=native -ffast-math`: relaxed FP semantics.

Table 2: Measured performance of the propagation kernel at `-O3 -march=native -ffast-math` (GCC 13.3, i9-13980HX, 50 000 repetitions).

d	Variant	ns/step	GFLOP/s	GB/s
3	AoS	62	10.4	25.5
3	SoA	35	18.7	45.7
3	AVX2	47	13.9	33.9
9	AoS	3 242	16.2	33.2
9	SoA	2 777	18.9	38.7
9	AVX2	3 623	14.5	29.7
27	AoS	295 028	14.4	28.9
27	SoA	248 562	17.1	34.3
27	AVX2	305 273	13.9	27.9

Assembly is captured via `objdump` and verified on Compiler Explorer [13]; interactive listings for each kernel variant are available in the project repository.

3.3. Benchmarking methodology

All timing uses `clock_gettime(CLOCK_MONOTONIC)`. We report mean wall time per step over $N = 50{,}000$ repetitions after a 10-step warm-up. FLOP/s and GB/s are derived analytically from the measured time.

4. Results

4.1. Roofline characterization

Table 2 presents measured performance for all three kernel variants at the best flag configuration (`-O3 -march=native -ffast-math`). At every system size, the SoA variant achieves the highest throughput.

The achieved bandwidth decreases from 45.7 GB/s ($d = 3$, L1) to 34.3 GB/s ($d = 27$, L3), consistent with the expected hierarchy: L1 delivers the highest bandwidth, and performance degrades as the working set spills to slower cache levels. The GFLOP/s remains roughly constant (17–19) because the increasing AI at larger d partially compensates for the lower bandwidth.

Table 3 shows the impact of compiler flags on the best layout (SoA). The most dramatic jump occurs between `-O3 -march=native` and `-O3 -march=native -ffast-math`, where `-ffast-math` enables auto-vectorization of complex arithmetic (Section 2.3).

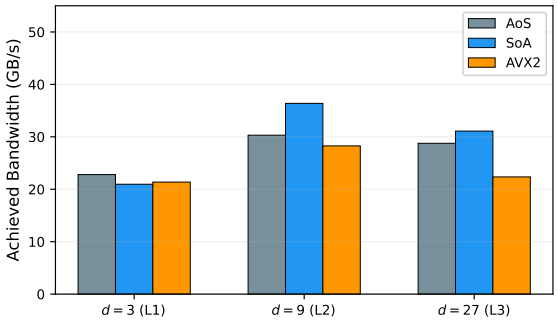

Figure 1: Achieved bandwidth (GB/s) for each layout variant at `-O3 -march=native -ffast-math`, grouped by system dimension. The x-axis labels indicate the cache level that holds the working set. SoA outperforms both AoS and hand-written AVX2 at every size. Bandwidth decreases from L1 to L3, consistent with the cache hierarchy.

4.2. *Compiler vectorization analysis*

The scalar baseline (`-O2 -fno-tree-vectorize`) generates an inner loop using 64-bit SSE scalar instructions operating on one real component at a time:

Listing 1: Inner loop of `lb_propagate_step` at `-O2 -fno-tree-vectorize` (12 instructions per complex element).

```
movsd   (%rbp),%xmm5          ; P[i,j].re
movsd   0x8(%rbx),%xmm3       ; vec[j].im
movsd   0x8(%rbp),%xmm1       ; P[i,j].im
movsd   (%rbx),%xmm2          ; vec[j].re
mulsd   %xmm1,%xmm0           ; P.im * v.im
mulsd   %xmm2,%xmm4           ; P.re * v.re
mulsd   %xmm2,%xmm8           ; P.im * v.re
subsd   %xmm0,%xmm4           ; acc_re partial
mulsd   %xmm3,%xmm0           ; P.re * v.im
addsd   %xmm8,%xmm0           ; acc_im partial
addsd   %xmm4,%xmm6           ; acc_re +=
addsd   %xmm0,%xmm7           ; acc_im +=
```

With `-O3 -march=native -ffast-math`, GCC auto-vectorizes to 256-bit AVX2 instructions processing two complex elements per iteration:

Table 3: Achieved bandwidth (GB/s) for SoA variant across compiler flag configurations.

Flags	d=3	d=9	d=27
`-O2 -fno-tree-vec.`	13.5	16.5	18.5
`-O3`	20.8	23.0	23.4
`-O3 -march=native`	21.0	18.4	28.0
`-O3 -march=nat. -ffast-math`	45.7	38.7	34.3

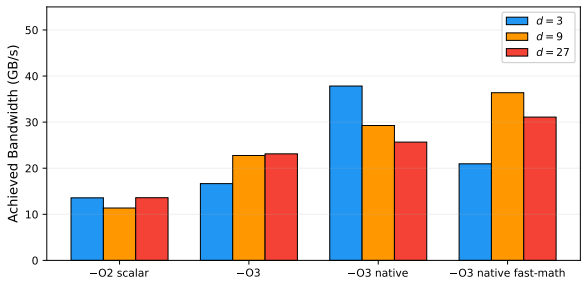

Figure 2: Achieved bandwidth (GB/s) for the SoA variant across compiler flag configurations and system dimensions. The `-ffast-math` flag produces the dominant speedup. Note the anomalous regression at $d = 9$ when adding `-march=native` without `-ffast-math` (see text).

Listing 2: Inner loop at `-O3 -march=native -ffast-math` (6 instructions per 2 complex elements). The `vfmsubadd231pd` instruction is the key enabler.

```
vpermilpd $0xf,(%rdx,%rax),%ymm0    ; P[j:j+2].im broadcast
vpermilpd $0x0,(%rdx,%rax),%ymm1    ; P[j:j+2].re broadcast
vmulpd    (%rcx,%rax),%ymm0,%ymm0   ; P.im * vec
vpermilpd $0x5,(%rcx,%rax),%ymm3    ; swap vec re<->im
vfmsubadd231pd %ymm3,%ymm1,%ymm0    ; P.re*vec_swap -/+ P.im*vec
vaddpd    %ymm0,%ymm2,%ymm2         ; accumulate
```

Key observations: (1) the scalar loop calls `__muldc3` on a branch (`jp`) for NaN handling, which the `-ffast-math` build eliminates entirely; (2) the vectorized loop processes 2 complex elements per iteration using 256-bit registers, a 2× throughput increase per instruction; (3) `vfmsubadd231pd` fuses the multiply, subtract, and add into a single instruction with 4-cycle latency and 0.5-cycle throughput [11].

Without `-ffast-math`, even `-O3 -march=native` falls back to VEX-encoded *scalar* FMA instructions (`vfmadd231sd`, `vfmsub231sd`); the compiler uses FMA but does not vectorize across complex elements, and the `__muldc3` fallback remains.

5. Discussion

The central finding is that the Lindblad propagation kernel is *memory-bound at all three system sizes* ($d = 3, 9, 27$), with measured arithmetic intensity 0.41–0.50 FLOP/byte against a DRAM ridge point of ~1.6 FLOP/byte. Performance is determined by how effectively the code feeds data from the relevant cache level to the floating-point units.

Layout matters more than intrinsics. The SoA variant consistently outperforms both the AoS and hand-written AVX2 variants (Table 2). At $d = 3$, SoA achieves 1.8× the throughput of AoS; at $d = 9$ and $d = 27$, the advantage is 1.2×. The hand-written AVX2 intrinsics on AoS layout are *slower* than auto-vectorized SoA at every size; the shuffle overhead of extracting real and imaginary parts from interleaved AoS data outweighs the benefit of explicit SIMD. This is a cautionary result for library authors: hand-writing SIMD for a suboptimal layout is a net performance loss.

Compiler flags dominate. The jump from `-O3 -march=native` to `-O3 -march=native -ffast-math` yields a 1.2–2.2× bandwidth improvement (Table 3). The root cause is that without `-ffast-math` (specifically `-fcx-limited-range`), GCC refuses to vectorize complex multiplication across SIMD lanes, falling back to scalar FMA with a `__muldc3` branch for NaN handling.

An anomalous result is visible in Table 3 and Fig. 2: at $d = 9$, adding `-march=native` to `-O3` *decreases* SoA bandwidth from 23.0 to 18.4 GB/s. Inspection of the assembly shows that `-march=native` switches the compiler to VEX-encoded scalar instructions (`vmulsd` vs. `mulsd`) without widening to 256-bit vectors. The VEX encoding provides no throughput benefit for scalar operations but may introduce overhead from upper-lane management. This regression disappears entirely once `-ffast-math` unlocks true SIMD vectorization.

Numerical accuracy. The `-ffast-math` flag relaxes IEEE 754 conformance: it enables FMA contraction (which changes rounding), assumes no NaN/Inf inputs (`-ffinite-math-only`), and uses a simplified complex multiply (`-fcx-limited-range`). For the Lindblad propagation kernel, the dominant numerical concern is accumulation error in the matvec inner product, which FMA contraction generally *improves* by reducing the number of rounding steps. We verified that all physical invariants (trace preservation, Hermiticity, diagonal positivity) hold to 10^{-8} tolerance with `-ffast-math` enabled. For applications requiring strict IEEE semantics, `-fcx-limited-range` alone suffices to enable vectorization without the broader `-ffast-math` relaxations.

Table 4: GRAPE-style piecewise-constant propagator chain timings at `-O3 -march=native -ffast-math`. Each landscape point requires building N propagators (one expm per segment) and chaining $N \times S$ matvec steps.

d	Segments	Build (ms)	Chain (ms)	Points/s
3	100	0.86	0.12	1016
9	50	58.8	4.0	15.9
27	20	13 414	232	0.1

Cache transitions are visible. Achieved bandwidth decreases monotonically from L1 (45.7 GB/s) through L2 (38.7 GB/s) to L3 (34.3 GB/s) at the best configuration, directly reflecting the cache hierarchy. This confirms that working set placement determines performance, and that for $d \leq 3$, the propagation step is fast enough (35 ns) that per-step overhead may dominate in a full GRAPE iteration.

GRAPE-style workloads are expm-dominated. Table 4 shows measured timings for piecewise-constant pulse simulation, where each pulse segment requires a separate propagator construction (one `expm` call per segment) followed by a chain of matvec steps. At all system sizes, the propagator build (expm) dominates the total cost. At $d = 3$ with a 100-segment pulse, the expm accounts for 88% of the per-point cost (0.86 ms build vs. 0.12 ms chain); at $d = 27$, it accounts for 98% (13.4 s build vs. 0.23 s chain). This means that for GRAPE robustness sweeps, optimizing the matvec kernel (the focus of this paper) yields diminishing returns once the propagation step is fast enough; the bottleneck shifts to the matrix exponential. Efficient expm implementations (e.g., exploiting Hamiltonian structure to reduce effective dimension) would have a larger impact on end-to-end GRAPE performance than further matvec optimization.

5.1. Recommendations for scientific simulation library authors

Based on our measurements, we offer the following concrete recommendations for implementations targeting small, fixed-size dense complex matrix operations ($n \lesssim 1000$):

1. **Precompute operators where possible.** For time-independent linear systems, compute the propagator (or transfer matrix) once and reuse it. The matrix exponential cost is amortized over thousands of applications.
2. **Use SoA layout for complex data.** Separate real and imaginary arrays enable clean 4-wide AVX2

vectorization without shuffle overhead. SoA outperforms AoS by 1.2–1.8× at these sizes. This recommendation applies to any computation over arrays of complex numbers, not only quantum simulation.

3. **Compile with `-O3 -march=native -ffast-math`.** The `-ffast-math` flag (or at minimum `-fcx-limited-range`) is essential for GCC to auto-vectorize complex arithmetic. Without it, the inner loop remains scalar.
4. **Consider specializing to fixed dimensions at compile time.** When the matrix size is known at compile time, the compiler can fully unroll small loops and eliminate bounds-check overhead.
5. **Align allocations to 64 bytes.** Use `aligned_alloc(64, ...)` for all matrix and vector buffers to enable aligned SIMD loads (`vmovapd` vs. `vmovupd`).
6. **Do not hand-write SIMD for interleaved complex data.** Our AVX2-intrinsics implementation on AoS layout is slower than compiler auto-vectorization of SoA layout at every size. Let the compiler vectorize clean data layouts rather than writing shuffles by hand.

6. Conclusion

We have presented a systematic microarchitectural analysis of a dense complex matrix-vector kernel at the small, fixed sizes relevant to near-term quantum control simulation. The kernel is memory-bound at all sizes ($d = 3, 9, 27$), with arithmetic intensity ≈ 0.5 FLOP/byte, well below the ridge point of modern CPUs. The best configuration (SoA data layout with `-O3 -march=native -ffast-math`) achieves 17–19 GFLOP/s (34–46 GB/s) on an Intel i9-13980HX, representing a 2–4× improvement over scalar AoS baselines.

The most actionable finding is that `-ffast-math` is not merely an optimization hint but a *prerequisite* for compiler auto-vectorization of complex arithmetic under GCC: without it, the inner loop remains scalar regardless of `-march=native`. Combined with SoA layout, this single flag change delivers the majority of the measured speedup.

We validated all qualitative findings (SoA dominance, `-ffast-math` prerequisite, memory-bound regime, hand-written AVX2 underperforming auto-vectorized SoA) on an AMD Ryzen 5 1600 (Zen 1, 2017) in addition to the Intel i9-13980HX results reported here; the conclusions are not vendor-specific.

While our test case is Lindblad master equation simulation, the recommendations (SoA layout, `-ffast-math`/`-fcx-limited-range`, cache-aware sizing, compiler-generated over hand-written SIMD) apply broadly to any scientific simulation code performing dense complex linear algebra at small, fixed matrix dimensions.

Future work includes extending this analysis to GPU kernels (where the bandwidth hierarchy shifts to global/shared/register memory), time-dependent systems (where the propagator must be recomputed at each step), and integration into a full GRAPE pipeline where propagation competes with gradient accumulation for cache residency.

Code availability

The complete source code, build scripts, benchmarking infrastructure, and analysis notebooks are available at `https://github.com/rylanmalarchick/lindblad-bench` under the MIT license. The repository includes CMake build configurations for all compiler flag variants reported in this paper.

Declaration of competing interest

The author declares no competing interests.

Acknowledgements

The author thanks Embry-Riddle Aeronautical University for support.

Appendix A. AI Disclosure

AI-assisted tools (Claude, Anthropic) were used for code development, debugging, and manuscript preparation. All scientific concepts, experimental design, analysis, and interpretation were performed by the author. The author takes full intellectual responsibility for all content.

References

[1] G. Lindblad, On the generators of quantum dynamical semigroups, Communications in Mathematical Physics 48 (2) (1976) 119–130. `doi:10.1007/BF01608499`.

[2] V. Gorini, A. Kossakowski, E. C. G. Sudarshan, Completely positive dynamical semigroups of n-level systems, Journal of Mathematical Physics 17 (5) (1976) 821–825. `doi:10.1063/1.522979`.

[3] J. Koch, T. M. Yu, J. Gambetta, A. A. Houck, D. I. Schuster, J. Majer, A. Blais, M. H. Devoret, S. M. Girvin, R. J. Schoelkopf, Charge-insensitive qubit design derived from the Cooper pair box, Physical Review A 76 (4) (2007) 042319. doi:10.1103/PhysRevA.76.042319.

[4] N. Khaneja, T. Reiss, C. Kehlet, T. Schulte-Herbrüggen, S. J. Glaser, Optimal control of coupled spin dynamics: design of NMR pulse sequences by gradient ascent algorithms, Journal of Magnetic Resonance 172 (2) (2005) 296–305. doi:10.1016/j.jmr.2004.11.004.

[5] R. Malarchick, When does numerical pulse optimization actually help? Error budgets, robustness tradeoffs, and calibration guidance for transmon single-qubit gates (2025). arXiv:2511.12799, doi:10.48550/arXiv.2511.12799.

[6] J. R. Johansson, P. D. Nation, F. Nori, QuTiP: An open-source Python framework for the dynamics of open quantum systems, Computer Physics Communications 183 (8) (2012) 1760–1772. doi:10.1016/j.cpc.2012.02.021.

[7] J. R. Johansson, P. D. Nation, F. Nori, QuTiP 2: A Python framework for the dynamics of open quantum systems, Computer Physics Communications 184 (4) (2013) 1234–1240. doi:10.1016/j.cpc.2012.11.019.

[8] P. Guilmin, R. Gautier, A. Le Bris, E. Music, D. E. Volpe, dynamiqs: an open-source Python library for GPU-accelerated differentiable simulation of quantum systems (2024). arXiv:2411.17166.

[9] S. Williams, A. Waterman, D. Patterson, Roofline: An insightful visual performance model for multicore architectures, Communications of the ACM 52 (4) (2009) 65–76. doi:10.1145/1498765.1498785.

[10] A. Fog, Optimizing Software in C++: An Optimization Guide for Windows, Linux, and Mac Platforms, Self-published, 2024, available at https://www.agner.org/optimize/.

[11] Intel Corporation, Intel intrinsics guide, https://www.intel.com/content/www/us/en/docs/intrinsics-guide/ (2024).

[12] N. J. Higham, The scaling and squaring method for the matrix exponential revisited, SIAM Journal on Matrix Analysis and Applications 26 (4) (2005) 1179–1193. doi:10.1137/04061101X.

[13] M. Godbolt, Compiler explorer, https://godbolt.org (2024).